

Week11

☰ 연습과제

01 K-평균 알고리즘 이해

K-평균(K-Means) 개요

- 군집화(Clustering)에서 **가장 일반적으로 사용되는 알고리즘**
- **군집 중심점(Centroid)** 을 기준으로 가장 가까운 데이터 포인트들을 하나의 군집으로 묶음
- 중심점은 **선택된 포인트들의 평균 위치**로 계속 이동

→ 더 이상 중심점 이동이 없으면 군집화 종료

K-평균 동작 과정

1. 초기 중심점 설정

- 만들고자 하는 **군집 개수(K)** 만큼 중심점을 임의의 위치에 설정
- 예: 2개 군집 → 중심점 2개 설정

2. 데이터 할당

- 각 데이터는 **가장 가까운 중심점**에 소속됨

→ “거리 기반 할당”

3. 중심점 이동

- 각 군집에 속한 데이터들의 **평균 위치**로 중심점을 이동

$$\mu_k = \frac{1}{n_k} \sum_{i=1}^{n_k} x_i$$

4. 재할당

- 중심점이 이동했으므로,
- 각 데이터가 **더 가까운 다른 중심점**이 있으면 소속 변경

5. 중심점 재계산

- 변경된 군집 기준으로 다시 평균 계산 → 중심점 이동

6. 종료 조건

- 더 이상 데이터 소속 변경이 없으면 종료
- 있으면 4~5 반복

→ 반복적(iterative) 알고리즘

K-평균의 장점

- 군집화에서 가장 많이 활용
 - 알고리즘이 쉽고 간결
-

K-평균의 단점

- 거리 기반 알고리즘 → 속성(feature)이 많으면 정확도 저하 가능
 - 필요 시 PCA 차원 축소 적용
 - 반복 횟수가 많으면 수행 시간 증가
 - 몇 개의 군집(K) 을 선택할지 결정하기 어려움
-

사이킷런 **KMeans** 클래스

- `sklearn.cluster.KMeans`

주요 파라미터

- `n_clusters` : 군집 개수
 - `init` : 초기 중심점 방식 (`k-means++`)
 - `max_iter` : 최대 반복 횟수
 - `random_state` : 난수 고정
-

02 군집 평가 (Cluster Evaluation)

군집 평가의 필요성

- 앞의 예제(make_blobs)는 **target label**이 존재해서 군집 결과와 실제 값을 비교할 수 있었음.
- 하지만 대부분의 군집화 데이터는 **정답(label)**이 없음.

→ 따라서 **군집이 얼마나 잘 형성됐는지 평가하는 별도 지표**가 필요함.

- 비지도학습 특성상 정확한 평가가 어렵지만,
 - 대표적으로 **실루엣 분석(Silhouette Analysis)** 사용
-

실루엣 분석 (Silhouette Analysis)

개요

- 각 군집이 **얼마나 잘 분리(separation)** 되어 있는지 측정
 - 좋은 군집화란:
 - 같은 군집 내부 데이터끼리는 가깝고
 - 다른 군집과는 멀리 떨어져 있는 것
-

실루엣 계수 (Silhouette Coefficient)

- 개별 데이터 포인트마다 계산되는 군집 품질 지표

각 데이터에 대해:

a(i)

- 자기 군집 내부 다른 데이터들과의 **평균 거리**
 - 작을수록 좋음 (같은 군집끼리 가까움)
-

b(i)

- 가장 가까운 **다른 군집**과의 평균 거리
 - 클수록 좋음 (다른 군집과 멀리 떨어짐)
-

실루엣 계수 정의

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

값의 의미

값	의미
1에 가까움	매우 좋은 군집
0 근처	군집 경계에 위치
음수(-)	잘못된 군집에 할당

sklearn 실루엣 함수

`silhouette_samples()`

- 각 데이터별 실루엣 계수 반환

`silhouette_score()`

- 전체 평균 실루엣 계수 반환

→ 일반적으로 값이 클수록 좋음

좋은 군집화 조건

1. 평균 실루엣 계수가 높아야 함

- 보통 0~1
- 1에 가까울수록 좋음

2. 군집별 편차가 작아야 함

- 특정 군집만 높고 다른 군집이 낮으면 안 됨
 - 모든 군집이 균형 있게 높아야 좋은 군집
-

붓꽃(Iris) 데이터 실습

평균 실루엣 계수

0.553

→ 전체적으로 무난한 군집화

군집별 평균

cluster	평균 실루엣
0	0.417
1	0.798
2	0.451

해석:

- 1번 군집은 매우 잘 분리됨
- 0, 2번은 상대적으로 낮음

→ 전체 평균만 보면 안 되고 **군집별 확인 필요**

군집 개수 최적화

- 평균 실루엣 값이 가장 높다고 항상 최적은 아님
- 각 군집이 고르게 좋은지 함께 봐야 함

→ 실루엣 시각화로 **최적 K 결정**

실루엣 시각화 (Silhouette Visualization)

목적

- 평균 실루엣 계수만 보면 군집 품질을 **완전히 판단하기 어려움**
- 각 군집별 **분포와 편차**를 함께 확인해야 함

→ 이를 위해 **실루엣 플롯(Silhouette Plot)** 사용

실루엣 플롯 해석 방법

x축

- 실루엣 계수 값
 - 범위: 1 ~ 1
-

y축

- 각 군집(cluster)에 속한 데이터들

막대의 길이

- 길수록 해당 데이터가 자기 군집에 잘 속함

막대 두께

- 해당 군집의 데이터 개수

좋은 실루엣 플롯 조건

1. 대부분 값이 양수(+)

- 음수가 많으면 잘못 군집화된 데이터 많음

2. 군집별 평균 길이가 길어야 함

- 실루엣 값이 클수록 좋음

3. 군집 간 두께가 지나치게 차이나면 안 됨

- 특정 군집에 데이터가 몰리면 불균형

군집 개수(K) 비교

K=2

- 군집 간 분리는 비교적 명확
- 하지만 너무 크게 묶일 수 있음

K=3

- 데이터 구조를 가장 자연스럽게 반영
- 평균 실루엣 계수 높음
- 군집 간 균형도 양호

→ 최적 K 후보

K=4

- 군집 수 증가로 일부 군집이 지나치게 작아짐
 - 음수 값 발생 가능
 - 과분할(over-clustering) 가능성
-

03 평균 이동 (Mean Shift)

평균 이동(Mean Shift) 개요

- K-Means와 유사하게 중심점을 기반으로 군집을 찾는 알고리즘
- 차이점:
 - K-Means → 평균 거리 중심
 - Mean Shift → 데이터 밀도가 높은 중심을 찾음

→ 확률 밀도 함수(probability density function) 를 이용하는 방식

KDE (Kernel Density Estimation)

개념

- 데이터 분포를 확률 밀도 함수 형태로 추정하는 방법
- 개별 데이터를 하나의 커널(kernel) 로 보고 합산

→ 데이터가 많이 몰린 곳 = 밀도 높음

커널 함수

- 대표적으로 가우시안 커널(Gaussian Kernel) 사용

데이터 주변에 종 모양 분포를 만든 뒤,
겹치는 정도로 밀도 계산

bandwidth (대역폭)

- 커널의 폭(width)을 결정하는 값
 - Mean Shift에서 가장 중요한 파라미터
-

bandwidth가 큰 경우

- 넓은 영역을 하나로 봄
 - → 군집 수 감소
-

bandwidth가 작은 경우

- 세밀하게 구분
 - → 군집 수 증가
-

Mean Shift 동작 방식

1. 초기 중심 설정

- 모든 데이터를 후보 중심으로 설정 가능
-

2. 주변 데이터 탐색

- bandwidth 안에 포함되는 데이터 탐색
-

3. 평균 위치 계산

- 주변 데이터의 평균 위치 계산
-

4. 중심 이동

- 계산된 평균 위치로 중심 이동
-

5. 반복

- 중심 이동이 없을 때까지 반복

→ 최종적으로 밀도 최대 지점(mode)에 도달

Mean Shift 특징

장점

- 군집 개수(K)를 지정할 필요 없음
 - 복잡한 형태의 군집 탐지 가능
-

단점

- 계산량이 많음
- 데이터 수가 많으면 느림
- bandwidth 설정에 민감

sklearn MeanShift 클래스

sklearn.cluster.MeanShift

주요 파라미터:

- `bandwidth` : 탐색 반경
- `bin_seeding` : 초기 seed 최적화

bandwidth 추정 함수

estimate_bandwidth()

- 적절한 bandwidth 자동 추정 가능

→ 실무에서 자주 사용

K-Means vs Mean Shift

구분	K-Means	Mean Shift
군집 개수 지정	필요(O)	불필요(X)
중심 기준	평균	밀도 최대점
속도	빠름	느림
복잡한 군집	어려움	가능

04 GMM (Gaussian Mixture Model)

GMM 개요

- Gaussian Mixture Model(가우시안 혼합 모델)
- 데이터를 여러 개의 가우시안 분포(Gaussian Distribution) 의 조합으로 가정

→ 각 군집이 하나의 **정규분포**를 따른다고 봄

기본 개념

K-Means는

- "가장 가까운 중심" 기준으로 **하나의 군집에만 속함 (Hard Clustering)**

GMM은

- 각 데이터가 **여러 군집에 속할 확률**을 가짐 (Soft Clustering)

예:

- 군집 A일 확률 70%
 - 군집 B일 확률 30%
-

정규분포(Gaussian Distribution)

특징

- 평균(mean) 중심
 - 분산(variance)에 따라 퍼짐 정도 결정
-

다변량 정규분포

- 여러 feature를 동시에 고려
- 타원형(elliptical) 분포 표현 가능

→ K-Means보다 복잡한 군집 표현 가능

GMM 구성 요소

1. 평균 (Mean)

- 각 군집 중심
-

2. 공분산 (Covariance)

- 군집의 방향성과 퍼짐 정도
-

3. 혼합 계수 (Mixing Coefficient)

- 각 군집이 전체에서 차지하는 비율
-

GMM 학습 방법: EM 알고리즘

E-step (Expectation)

- 각 데이터가 각 군집에 속할 확률 계산
-

M-step (Maximization)

- 계산된 확률 기반으로
 - 평균
 - 공분산
 - 혼합 계수업데이트
-

반복

- 수렴(convergence)할 때까지 반복
-

sklearn GaussianMixture

```
sklearn.mixture.GaussianMixture
```

주요 파라미터:

- `n_components` : 군집 개수
 - `covariance_type` : 공분산 형태 지정
 - `full`
 - `tied`
 - `diag`
 - `spherical`
-

covariance_type 의미

타입	의미
full	각 군집마다 독립 공분산
tied	모든 군집 동일 공분산
diag	대각선만 사용
spherical	구형 분포

GMM 장점

- 타원형 군집 표현 가능
- K-Means보다 유연
- soft clustering 가능

GMM 단점

- 계산량 큼
- 초기값에 영향 받을 수 있음

K-Means vs GMM

구분	K-Means	GMM
군집 방식	Hard	Soft
군집 형태	구형	타원형
계산 속도	빠름	느림
유연성	낮음	높음

05 DBSCAN (Density Based Spatial Clustering of Applications with Noise)

DBSCAN 개요

- 밀도 기반(Density-based) 군집화 알고리즘
- 데이터가 밀집된 영역을 하나의 군집으로 판단

→ 중심점 기반(K-Means)과 다른 방식

특징

- 군집 개수를 **사전에 지정할 필요 없음**
 - 복잡한 형태의 군집 탐색 가능
 - 이상치(Noise) 자동 탐지 가능
-

핵심 파라미터

eps (입실론)

- 한 데이터 주변의 **반경(radius)**

→ 이 거리 안에 있는 점들을 이웃(neighbor)으로 판단

min_samples

- 군집으로 인정하기 위한 **최소 데이터 개수**

예:

- min_samples = 5
 - 반경 안에 5개 이상 있으면 군집 형성
-

데이터 유형

Core Point (핵심점)

- eps 반경 내 데이터 수 $\geq$ min_samples

→ 군집 중심 역할

Neighbor Point

- Core Point 주변 데이터
-

Border Point (경계점)

- 자기 주변 밀도는 낮지만
- Core Point 근처에 존재

→ 군집에 포함됨

Noise Point (이상치)

- 어떤 군집에도 속하지 못하는 점

→ outlier 처리

DBSCAN 동작 방식

1. 임의의 데이터 선택

2. eps 반경 탐색

- 주변 데이터 개수 확인
-

3. Core Point 여부 판단

- 조건 만족 시 군집 생성
-

4. 인접 데이터 확장

- 연결된 Core Point 계속 탐색

→ 군집 확장

5. 반복

- 모든 데이터에 대해 수행
-

sklearn DBSCAN

sklearn.cluster.DBSCAN

주요 파라미터:

- `eps`
 - `min_samples`
 - `metric`
-

labels_

- 군집 결과 저장

특징:

- 1 → Noise Point

DBSCAN 장점

- 군집 개수 지정 불필요
- 이상치 자동 탐지
- 복잡한 모양 가능

DBSCAN 단점

- eps 설정 어려움
- 데이터 밀도 차이가 크면 성능 저하

K-Means vs DBSCAN

구분	K-Means	DBSCAN
기준	중심점	밀도
K 지정	필요	불필요
이상치 처리	어려움	가능
군집 형태	구형	자유로운 형태

활용

- 공간 데이터 분석
- 이상치 탐지
- 이미지 처리
- 위치 기반 서비스

06 고객 세그먼테이션 (Customer Segmentation)

고객 세그멘테이션 개요

- 고객(Customer) 을 다양한 기준으로 분류하는 기법
 - CRM(Customer Relationship Management)과 마케팅의 핵심 요소
-

고객 분류 기준

고객은 여러 기준으로 분류 가능

- 지역
- 성별
- 결혼 여부
- 소득
- 직업

하지만 실제 비즈니스에서는 **구매 행동 기반 정보**가 더 중요

- 구매 금액
- 구매 빈도
- 최근 구매 시점
- 구매 주기
- 선호 상품

→ **타겟 마케팅(Target Marketing)** 에 활용

RFM 기법

고객 세분화의 대표적 기준

Recency (R)

- 최근 구매일로부터 현재까지의 기간

→ 최근 구매했수록 높은 가치

Frequency (F)

- 구매 횟수

→ 많이 구매할수록 우수 고객

Monetary Value (M)

- 총 구매 금액

→ 지출이 많을수록 중요 고객

데이터 세트

Online Retail Data Set

- UCI 제공
- 영국 온라인 쇼핑몰 거래 데이터

특징:

- 거래 건수: **541,909건**
 - 컬럼 수: **8개**
-

데이터 전처리

Null 제거

- `CustomerID` 결측치 제거

→ 고객 식별 불가 데이터 삭제

오류 데이터 제거

조건:

- `Quantity > 0`
- `UnitPrice > 0`

이유:

- 음수 Quantity는 반품 데이터
 - 분석 목적상 제외
-

파생 변수 생성

총 구매 금액 생성:

```
sale_amount = Quantity × UnitPrice
```

RFM 데이터 생성

`groupby(CustomerID)` 수행

aggregation:

- `InvoiceDate → max()` → Recency
- `InvoiceNo → count()` → Frequency
- `sale_amount → sum()` → Monetary

Recency 변환

- 가장 최근 구매일 기준으로 날짜 차이 계산

```
Recency = 기준일 - 최근 구매일
```

→ “며칠 전 구매했는가”

데이터 분포 확인

`describe()` 결과:

특징:

- 왜곡(skewness) 심함
- 극단값(outlier) 존재

예:

- Frequency max = 7847
- Monetary max = 259657

→ 일부 고객이 평균을 왜곡

Standard Scaling 후 K-Means

사용:

- `StandardScaler()`

- `KMeans(n_clusters=3)`

결과:

- 실루엣 점수 ≈ 0.592

겉보기엔 나쁘지 않음

문제점

군집별 분포를 보면

- 특정 군집에 데이터가 몰림
- 균형이 좋지 않음

→ 왜곡 데이터 영향

로그 변환(Log Transform)

목적:

- 큰 값 축소
- 왜곡 완화

적용:

$$x' = \log(1+x)$$

로그 변환 후 재군집화

다시:

- StandardScaler
- KMeans 수행

결과:

- 실루엣 점수 ≈ 0.303

→ 숫자는 낮아짐

하지만 군집 분포가 더 균일

- 각 군집 크기 균형 개선
 - 실제 비즈니스 활용도 증가
-

결론

왜곡된 데이터에서는

바로 군집화 X

→ 로그 변환 등 데이터 변환 먼저 수행

즉,

데이터 정제

→ 스케일링

→ 왜곡 확인

→ 로그 변환

→ 군집화

순서가 중요